# SPECTRE: An Event-Driven Vision-Language Tactical Coach for Apex Legends

AI Systems Architecture — Midterm Progress Report
Utah State University

## 1 Introduction

Competitive gameplay analysis is fundamentally a problem of contextual reasoning. In Apex Legends, the significance of an action depends on the player's position, available resources, teammate status, enemy threats, and the events that immediately precede or follow it. Performance statistics can summarize an engagement, but they seldom explain whether a different decision would have produced a better outcome.

SPECTRE addresses this limitation by combining structured visual perception with vision-language reasoning. Rather than requiring a general-purpose VLM to interpret every element of a gameplay frame, the system extracts selected HUD information using conventional computer vision and provides that information as temporal context during multimodal analysis.

The project began with an architectural question: *Can structured gameplay observations improve the reliability and usefulness of VLM-generated tactical feedback?*

At the midterm milestone, SPECTRE has progressed from a proposed architecture to a functional offline prototype. The implemented pipeline processes gameplay recordings, identifies screen states, recognizes squad legends and equipped weapons, estimates shield strength, constructs timestamped observations, and performs sequential VLM analysis. The model produces structured descriptions of gameplay events, potential mistakes, recommendations, and uncertainties.

Initial experiments confirm that the components can operate together. They also expose important limitations. The VLM sometimes misinterprets ammunition mechanics, treats unavailable measurements as evidence of player incapacitation, and fails to reconstruct complete engagement outcomes. These findings provide a concrete basis for revising the architecture toward stronger temporal reasoning and game-specific knowledge.

## 2 System Architecture

SPECTRE uses a two-stage processing architecture that separates visual measurement from multimodal interpretation.

### 2.1 Offline Perception Pipeline

The first stage, implemented in `extract.py`, processes recorded gameplay using OpenCV. Source recordings have a resolution of $3840 \times 1600$ pixels. Each sampled frame is evaluated using screen-specific detectors configured through a common JSON file.

The perception stage produces a chronological match record containing frame indices, timestamps, and extracted HUD observations. Its output is independent of the VLM, allowing individual detectors to be tested without repeating multimodal inference.

### 2.2 Vision-Language Analysis Pipeline

The second stage, implemented in `analyze.py`, reads the previously generated match record and retrieves the corresponding images directly from the original recording.

Frames and their associated HUD observations are combined into chronological batches and submitted to a locally hosted Qwen3.5 vision-language model through the `llama.cpp` multimodal interface.

The model is instructed to identify significant events, analyze positioning and coordination, and propose improvements grounded in visible evidence. Its output is serialized into a structured JSON report.

The separation between these stages offers two practical benefits. First, the expensive VLM need not repeat deterministic HUD recognition. Second, the same extracted match data can support different prompts, models, and analysis policies without rerunning the perception pipeline.

# 3 Implementation

## 3.1 HUD Configuration and Preprocessing

A major implementation challenge is the complexity of Apex Legends' interface. Relevant information appears in small screen regions and may change appearance with shield type, weapon selection, player status, or the currently open menu.

SPECTRE addresses this through a preprocessing notebook that records regions of interest (ROIs) and visual references in `hud_regions.json`. The configuration separates screen-specific extraction parameters from the runtime logic.

This calibration process is necessary because a detector that performs well on normal gameplay may produce meaningless results when the same screen coordinates contain a death-box menu, inventory panel, or map overlay.

## 3.2 Screen-State Classification

The system recognizes five explicit non-gameplay screen states: inventory, map, spectating, death-box interaction, and match summary. Each is associated with a distinctive interface template and a fixed-position ROI.

Normal gameplay is identified when these explicit interfaces are not detected and the player's legend portrait provides sufficient evidence of an active gameplay HUD.

A conservative classification policy is applied. If no candidate satisfies the required similarity threshold, or multiple candidates are insufficiently distinguishable, the screen is classified as `unknown`.

Gameplay-specific extraction is performed only for frames classified as normal gameplay.

This mechanism resolves a failure observed during development: before screen classification was introduced, the shield detector occasionally interpreted death-box interface elements as equipped shields. The updated pipeline avoids applying that detector to recognized death-box screens.

## 3.3 Legend Recognition

Legend recognition uses position-specific template matching. Separate reference collections are maintained for the local player and two teammates because portraits may have different sizes or appearances across the three HUD positions.

The current implementation contains 28 reference portraits for each position. Templates are loaded once, converted to grayscale, and compared against the corresponding frame ROI using normalized cross-correlation.

For candidate template $T$ and observed region $I$, the detector selects

$$\hat{\ell} = \arg\max_{\ell} \mathrm{NCC}(I, T_\ell),$$

subject to a minimum similarity threshold.

In an inspected sequence, the detector repeatedly identified Wattson, Catalyst, and Conduit in their correct squad positions. Accepted similarity scores commonly exceeded 0.93, with several observations above 0.98.

These scores demonstrate strong template correspondence in the tested footage; they should not be interpreted as dataset-wide recognition accuracy.

## 3.4 Shield Classification and Estimation

Shield extraction combines calibrated color references with horizontal fill measurement. The current configuration includes six visual categories: white, blue, and purple shields, each with a corresponding Amped variant.

Because the three squad positions use different screen coordinates, separate reference regions are recorded for each player.

The runtime detector evaluates candidate shield colors and tracks matching pixels from the left edge of the shield bar toward the right. Small interruptions are tolerated to account for segmented visual elements.

The estimated shield strength is

$$\hat{s} = \frac{w_{\text{filled}}}{w_{\text{reference}}} s_{\max},$$

where $s_{\max}$ denotes the nominal capacity associated with the recognized shield type.

The left-anchored approach reduces the influence of isolated matching pixels outside the filled region. Nevertheless, visual effects, bar boundaries, and changes in shield appearance can still affect the estimate. Amped-shield classification remains limited when a partially depleted Amped shield is visually indistinguishable from a normal one.

Shield values are therefore treated as approximate observations rather than verified ground truth.

### 3.5 Weapon Recognition

Equipped weapon names are extracted from the corresponding HUD region using Tesseract OCR and a restricted vocabulary of Apex Legends weapons.

The initial implementation also attempted to recognize magazine and reserve ammunition and associate the observed values with the active weapon.

These experiments revealed two systematic errors. First, dimmed ammunition placeholders were occasionally recognized as numerical values. Second, the active weapon could be identified incorrectly, causing ammunition readings to be associated with the wrong weapon.

For the midterm prototype, the reliable component—weapon identity—has been retained. Ammunition extraction is excluded from the primary structured observations until its validity can be established.

This decision reflects an important design principle: an unavailable measurement is preferable to an incorrect measurement presented as reliable context.

### 3.6 Temporal Match Representation

Each sampled frame contributes an observation containing its timestamp, source frame index, recognized screen state, and available HUD information.

The observations are stored in a match-level JSON timeline. This structure preserves the sequence of changing gameplay conditions and supports direct alignment with the original recording.

An important distinction is maintained between measured information and missing information. A value of `null` indicates that the field was not observed; it does not imply that the associated resource is empty or that a player has been knocked down.

The current system records observations chronologically, but a more comprehensive state tracker—including last-known values, validity timestamps, and explicit state transitions—remains under development.

## 4 Experimental Evaluation

### 4.1 Experimental Setup

The prototype was evaluated using recorded Apex Legends gameplay and a locally hosted Qwen3.5 vision-language model.

The initial configuration samples gameplay at one frame per second and submits batches of ten chronologically ordered frames. Each image is accompanied by its corresponding structured HUD observation.

The VLM returns a JSON object containing a segment summary, timestamped events, potential mistakes, suggested improvements, and unresolved uncertainties.

Streaming output is enabled to observe generation behavior, while execution logs retain screen classifications, detector scores, and model token usage.

The present results constitute functional and qualitative evaluation on selected gameplay segments rather than a comprehensive accuracy benchmark.

### 4.2 Non-Combat Sequence

The first experiment analyzed a ten-second segment covering movement, looting, and inventory interaction.

The model successfully recognized the player's movement from an elevated position toward cover, interaction with loot, opening of the inventory screen, and subsequent repositioning. It also acknowledged that no visible combat occurred during the analyzed interval.

The experiment established that structured observations and their corresponding images could be combined into a coherent chronological analysis.

One measured request contained approximately 17,600 prompt tokens, with approximately 20,800 total tokens consumed. This illustrates the substantial computational cost of multimodal interpretation even for a short sequence.

However, the model also produced inaccurate ammunition recommendations, demonstrating that successful visual recognition does not necessarily imply correct knowledge of game mechanics.

### 4.3 Combat Sequence

A second experiment examined the interval from 540 to 549 seconds, which contained an active engagement.

The model recognized changes in positioning, incoming damage, a downed teammate, and a visible enemy knockdown. It also identified that the player continued fighting after receiving substantial damage.

However, the analysis failed to recognize the complete engagement outcome. Although the player eliminated an opposing squad, the generated report identified only an individual knockdown.

The model consequently recommended retreating or prioritizing a teammate revive without adequately considering the successful elimination of the immediate enemy threat.

This failure is particularly relevant to SPECTRE's purpose: a tactical coach must understand the development and resolution of an engagement before judging the decisions made within it.

## 5 Discussion

The initial results reveal three issues that cannot be resolved through HUD extraction alone.

### 5.1 Recognition Is Not Game Understanding

In one experiment, the VLM described a Mastiff with five loaded rounds as critically low on ammunition, despite five rounds representing a full magazine in the tested configuration.

It also confused ammunition types and incorrectly treated the Flatline's observed reserve ammunition as a fixed maximum capacity.

These errors indicate that generic multimodal reasoning is insufficient when tactical conclusions depend on specialized weapon mechanics.

SPECTRE therefore requires an explicit, versioned knowledge source describing weapon capacities, ammunition types, legend abilities, and other game-specific conventions.

### 5.2 Missing Data Must Not Become False Evidence

During combat analysis, the model repeatedly interpreted `health: null` as evidence that the player was downed. This contradicted the frames, which showed continued weapon use and engagement.

The problem was not merely an inaccurate visual prediction. It reflected a semantic mismatch between the extraction schema and the model's interpretation of that schema.

Future observations will distinguish unavailable measurements, last-known values, and confirmed states. The model must treat an absent measurement as insufficient evidence rather than a negative event.

### 5.3 Tactical Reasoning Requires Temporal Context

A ten-second batch sampled at one frame per second provides only ten visual observations. Rapid combat events—including weapon switches, knockdowns, eliminations, and movement between cover positions—may occur entirely between these samples.

The missed squad wipe illustrates this limitation. The model recognized an individual knockdown but lacked sufficient temporal evidence, or failed to connect the available evidence, to reconstruct the engagement outcome.

Another limitation arises at batch boundaries: a fight may begin in one batch and end in the next. A short natural-language summary of previous observations does not necessarily preserve every relevant event.

The next architecture will therefore introduce fight-aware sampling and explicit event memory.

## 6 Future Work

The next development stage prioritizes four extensions.

**Game-specific knowledge integration.** A structured knowledge base will describe weapons, legends, resource mechanics, and relevant HUD conventions. Only entries relevant to the current match segment will be supplied to the VLM.

**Fight-aware temporal sampling.** The system will investigate higher sampling rates during combat, initially targeting approximately five frames per second. Surrounding context will be retained to support interpretation of both engagement initiation and outcome.

**Explicit state and event tracking.** A separate state-management component will maintain validated observations, last-known resource values, and timestamped events such as damage, knockdowns, and weapon changes.

**Domain adaptation and evaluation.** Annotated Apex gameplay examples will be collected to evaluate domain-specific errors and investigate supervised fine-tuning after establishing a knowledge-guided baseline.

Recognition of tactical abilities, inventory contents, and audio events is also planned. The longer-term objective remains event-driven, deadline-aware coaching, in which late or insufficiently supported advice is suppressed.

## 7 Conclusion

SPECTRE has reached its first functional milestone: a complete offline pipeline that combines deterministic HUD perception with sequential vision-language analysis.

The prototype demonstrates reliable integration of video sampling, screen-state identification, selected HUD detectors, temporal JSON observations, and locally hosted multimodal inference.

Its experiments also establish an important architectural lesson. Extracting visual facts is necessary but insufficient for tactical coaching. Reliable feedback requires game-specific knowledge, explicit treatment of uncertainty, and temporal reasoning that connects individual actions to engagement outcomes.

The next phase will address these limitations while preserving the modular architecture established by the current implementation.